

Intelligent E-Governance Citizen Service Request Management System

Student Name: S.Suvikumar

Register Number: 192424262

Department: AI&DS

Abstract

Traditional aquaculture and fisheries farms largely depend on manual observation to track water-quality parameters such as temperature, pH, dissolved oxygen, and turbidity. Manual monitoring is slow, error-prone, and often fails to detect abnormal pond conditions before they affect fish health and farm productivity. This report presents the design of a Smart Fisheries Monitoring and Aquaculture Management System that continuously collects environmental readings from simulated sensors, transmits them through a gateway layer to a backend REST API, stores the readings in a relational database, and presents live and historical information through a web-based dashboard. An alert engine compares incoming readings against safe threshold ranges and flags abnormal conditions so that farm administrators can respond quickly.

The assignment focuses on two supporting engineering practices that make such a system maintainable and deployable: Git for distributed version control and Docker for containerized deployment. Git is used to organize the source code into a structured repository with a disciplined branching strategy (main, develop, and feature branches) so that multiple contributors can work in parallel without disrupting a stable codebase. Docker is used to package the frontend, backend, and database components into independent, reproducible containers that are orchestrated together using Docker Compose, removing the need to manually install and configure dependencies on every machine.

The report documents the system's layered software architecture, repository structure, Git workflow and branching model, Dockerfile and Compose configuration, deployment and testing procedure, and a quality-attribute analysis covering performance, scalability, security, reliability, and maintainability. The expected benefit of the proposed system is a more reliable, collaborative, and portable development process that can be extended toward real IoT hardware and cloud deployment in future iterations.

Table of Contents

Abstract.....	2
1. Introduction.....	5
2. Problem Analysis	5
3. System Requirements.....	7
4. Project Structure Design	8
5. Software Architecture	9
6. Git Repository Initialization.....	10
7. Git Branching Strategy	12
8. Version Control Workflow	12
9. Commit History Analysis	13
10. Docker Environment Design.....	14
11. Dockerfile Development	14
12. Docker Compose Design	15
13. Container Deployment and Testing	16
14. Architecture and Component Interaction.....	18
15. Quality Attribute Analysis	19
16. Benefits of Git and Docker	19
17. Challenges and Improvements.....	20
18. Results and Expected Outcomes	21
19. Conclusion	21
20. References.....	22
Rubric Alignment.....	23

List of Figures

- Figure 1: Layered Software Architecture
- Figure 2: Project Repository Structure
- Figure 3: Git Workflow (Working Directory to Remote Repository)
- Figure 4: Git Branching Strategy
- Figure 5: Docker Container Architecture
- Figure 6: Data Flow and Component Interaction
- Figure 7: Deployment Workflow

List of Tables

- Table 1: Functional Requirements
- Table 2: Non-Functional Requirements
- Table 3: Hardware Requirements
- Table 4: Software Requirements
- Table 5: Repository Folder and File Description
- Table 6: Dockerfile Instruction Explanation
- Table 7: Docker Compose Configuration Elements
- Table 8: Container Testing Plan
- Table 9: Quality Attribute Analysis
- Table 10: Comparison – Without vs. With Git/Docker
- Table 11: Rubric Alignment

1. Introduction

Fisheries and aquaculture management refers to the set of practices used to raise, monitor, and maintain fish or other aquatic organisms in a controlled environment such as ponds, tanks, or cages. The success of aquaculture depends heavily on maintaining stable water-quality conditions, since fish are highly sensitive to changes in temperature, pH, dissolved oxygen, and turbidity. Even short periods of poor water quality can lead to stress, disease, or large-scale fish mortality.

Traditionally, farm workers monitor these parameters manually using handheld test kits and visual inspection at fixed intervals. This approach has several drawbacks: readings are taken infrequently, human error can occur during measurement or recording, and abnormal conditions occurring between inspection rounds may go unnoticed until visible damage has already occurred. As farms grow larger, manual monitoring becomes increasingly difficult to scale.

Digital monitoring addresses these limitations by using sensors to continuously measure environmental parameters and software systems to record, analyze, and present this information in real time. The Internet of Things (IoT) plays a central role by allowing distributed sensor devices to communicate readings to a central system without manual intervention. Once data reaches a software backend, it can be validated, stored, visualized on a dashboard, and compared against safe thresholds to automatically raise alerts.

This assignment develops the Smart Fisheries Monitoring and Aquaculture Management System as a case study for applying two essential software engineering practices: Git and Docker. Git provides a structured, auditable way to manage the evolution of source code across multiple contributors and features, while Docker ensures that the application's frontend, backend, and database run consistently regardless of the underlying operating system or installed dependencies. Together, these tools support a development workflow that is collaborative, reproducible, and suitable for eventual deployment to a production or cloud environment.

2. Problem Analysis

2.1 Problem Statement

Traditional aquaculture farms depend on manual monitoring of key environmental parameters, including temperature, pH, dissolved oxygen (DO), turbidity, feeding schedules, and general pond or tank conditions. Because these checks are performed periodically rather than continuously, there can be a significant delay between the onset of an abnormal condition and its detection by farm staff.

This delay is problematic because water-quality deterioration can progress quickly. A drop in dissolved oxygen during the night, for example, may go unnoticed until morning, by which time fish stress or mortality may already have occurred. Without a historical record of readings, it is also difficult to identify long-term trends or recurring problems, making it harder to plan preventive action. The lack of an integrated software system further means that data about feeding, pond conditions, and fish health is often scattered across paper logs rather than a single, searchable source of truth.

2.2 Project Objectives

1. Design a layered software architecture for continuous aquaculture monitoring.
2. Simulate sensor readings for temperature, pH, dissolved oxygen, and turbidity.
3. Provide a backend REST API to receive, validate, and store sensor data.

4. Implement an alert engine that flags readings outside safe threshold ranges.
5. Provide a web dashboard for real-time and historical data visualization.
6. Maintain a structured, version-controlled Git repository with a disciplined branching strategy.
7. Containerize the frontend, backend, and database using Docker for portable deployment.
8. Use Docker Compose to orchestrate multi-container deployment with a single command.
9. Support pond and feeding-record management for farm administrators.
10. Establish a foundation that can later be extended to real IoT hardware and cloud hosting.

2.3 Functional Requirements

Table 1: Functional Requirements

Req. ID	Functional Requirement	Description
FR-01	User Login	Administrators and farm staff authenticate before accessing the dashboard.
FR-02	Dashboard	Displays live and historical sensor readings in a unified view.
FR-03	Temperature Monitoring	Captures and stores water temperature readings at regular intervals.
FR-04	pH Monitoring	Captures and stores pH level readings for each pond or tank.
FR-05	Dissolved Oxygen Monitoring	Captures and stores DO readings to detect oxygen depletion.
FR-06	Turbidity Monitoring	Captures and stores turbidity readings to assess water clarity.
FR-07	Data Storage	Persists all sensor readings and events in a relational database.
FR-08	Alert Generation	Automatically raises alerts when readings breach safe thresholds.
FR-09	Pond Management	Allows administrators to register and manage ponds or tanks.
FR-10	Feeding Management	Records feeding schedules and quantities per pond.
FR-11	Historical Data	Provides access to past readings for trend analysis.
FR-12	Administration	Supports user and system-configuration management.

2.4 Non-Functional Requirements

Table 2: Non-Functional Requirements

Attribute	Requirement Description
Performance	The API should respond to dashboard requests within an acceptable time under normal load.
Security	User access must be authenticated; credentials and environment variables must not be hard-coded.
Scalability	The architecture should allow additional sensors, ponds, or containers to be added with minimal change.
Reliability	The system should consistently record readings and generate alerts without data loss.
Availability	Core services should remain accessible during normal farm operating hours.
Maintainability	Code should be modular and organized so that features can be updated independently.

Attribute	Requirement Description
Portability	The application should run consistently across different machines using Docker containers.
Usability	The dashboard should present information clearly for non-technical farm staff.

2.5 Expected Outcomes

The completed system is expected to demonstrate an organized Git repository with a clear branching history, a working set of Docker containers that can be built and deployed with a single Docker Compose command, and a functioning (simulated) data pipeline from sensor input to dashboard visualization, including automated alert generation for abnormal readings. These outcomes together demonstrate the practical value of version control and containerization in a real-world style software project.

3. System Requirements

3.1 Hardware Requirements

For this assignment, sensor hardware is simulated in software rather than physically implemented. The table below lists the hardware that a real-world deployment of this system would use; the sensor simulator module (`sensor_simulator/sensor.py`) generates representative values in place of these physical devices for development and demonstration purposes.

Table 3: Hardware Requirements (Target Deployment)

Component	Purpose
Computer / Laptop	Development machine used to build, run, and test the system.
Temperature Sensor	Measures water temperature within ponds or tanks.
pH Sensor	Measures the acidity/alkalinity of the water.
Dissolved Oxygen Sensor	Measures the amount of oxygen available to aquatic life.
Turbidity Sensor	Measures water clarity / suspended particle concentration.
Microcontroller / IoT Gateway	Collects sensor readings and forwards them to the backend.
Network Connection	Provides connectivity between the gateway and the backend server.

3.2 Software Requirements

Table 4: Software Requirements

Software	Role in the Project
Git	Distributed version control for source-code management.
GitHub	Remote repository hosting and collaboration.
Docker	Containerization of application services.
Docker Compose	Multi-container orchestration and deployment.
Python	Backend application logic and sensor simulation.

Software	Role in the Project
MySQL	Relational database for storing sensor and farm data.
HTML / CSS / JavaScript	Frontend dashboard structure, styling, and interactivity.
Web Browser	Client used to access the dashboard.
VS Code	Source-code editor used during development.

Note: sensor hardware and readings referenced throughout this report are simulated for development and academic demonstration; no physical sensor hardware was procured or installed as part of this assignment.

4. Project Structure Design

The repository is organized by responsibility, separating backend, frontend, database, sensor simulation, tests, and documentation into independent folders. This structure keeps each concern isolated and makes it easier for multiple contributors to work on different parts of the system without conflict.

Figure 2: Project Repository Structure

```

smart-fisheries/
|
|-- backend/
|   |-- app.py
|   |-- requirements.txt
|   |-- routes/
|   |-- models/
|   `-- services/
|
|-- frontend/
|   |-- index.html
|   |-- dashboard.html
|   |-- css/
|   `-- js/
|
|-- database/
|   |-- schema.sql
|   `-- seed.sql
|
|-- sensor_simulator/
|   `-- sensor.py
|
|-- tests/
|   |-- test_api.py
|   `-- test_sensor.py
|
|-- docs/
|   |-- architecture.png
|   |-- git-workflow.png
|   `-- docker-architecture.png
|
|-- .gitignore
|-- README.md
|-- Dockerfile
|-- docker-compose.yml

```



```
`-- .env
```

Figure 2: Project Repository Structure

Table 5: Repository Folder and File Description

Path	Purpose
backend/	Flask/Python application: API routes, data models, and business-logic services.
frontend/	Static HTML/CSS/JS files that render the monitoring dashboard.
database/	SQL schema and seed scripts used to initialize the MySQL database.
sensor_simulator/	Generates simulated sensor readings for development and testing.
tests/	Automated tests for the API and sensor simulator.
docs/	Architecture and workflow diagrams referenced in this report.
.gitignore	Lists files/folders Git should not track (e.g., env files, caches).
README.md	Project overview, setup instructions, and usage guide.
Dockerfile	Build instructions for the backend application image.
docker-compose.yml	Defines and links the frontend, backend, and database containers.
.env	Stores environment-specific configuration values (kept out of version control).

5. Software Architecture

The system follows a layered architecture, a widely used architectural style that separates concerns into distinct horizontal layers, each responsible for a specific part of the overall processing pipeline. This style was chosen because it maps naturally onto the flow of data in a monitoring system: from physical sensing, through communication and processing, to storage and presentation.

Sensor / IoT Layer: Represents the physical (or simulated) sensors that measure temperature, pH, dissolved oxygen, and turbidity.

Communication Layer: The sensor gateway that collects readings from sensors and forwards them to the backend over the network.

Application / Backend Layer: The Flask-based REST API and alert engine that validate, process, and route sensor data.

Database Layer: The MySQL database that persists sensor readings, ponds, feeding records, users, and alerts.

Presentation Layer: The web dashboard through which farmers and administrators view and interact with the data.

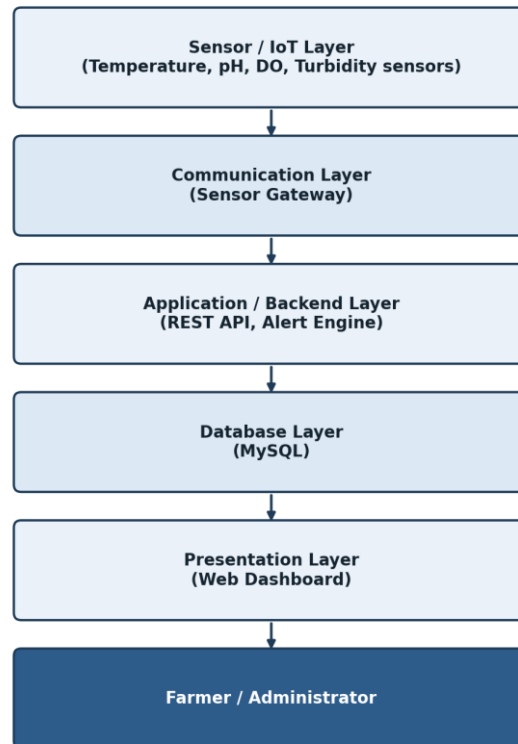


Figure 1: Layered Software Architecture

Each layer only communicates with the layer directly adjacent to it, which keeps responsibilities isolated: the backend does not need to know how sensor readings were physically captured, and the dashboard does not need to know how the database is implemented. This isolation makes the system easier to test, maintain, and extend — for example, real sensor hardware could later replace the simulator without requiring any change to the backend, database, or dashboard layers.

6. Git Repository Initialization

Git is a distributed version control system that records the history of changes made to a project's source code. Every contributor works with a full copy of the repository history, which allows changes to be tracked, compared, reverted, and merged in a controlled manner. Git was chosen for this project because it supports offline work, safe experimentation through branching, and clear accountability through commit history — all of which are valuable in a multi-feature project like this one.

6.1 Initializing the Repository

```

mkdir smart-fisheries
cd smart-fisheries
git init
git add .
git commit -m "Initial project setup"
git branch -M main
git status
git log --oneline
  
```

mkdir smart-fisheries / cd smart-fisheries — Creates and enters the project's root folder.

git init — Initializes a new, empty Git repository by creating the hidden .git directory.

git add . — Stages all current files so they will be included in the next commit.

git commit -m "..." — Saves the staged changes as a new commit with a descriptive message.

git branch -M main — Renames the default branch to main.

git status — Shows the current state of the working directory and staging area.

git log --oneline — Displays a condensed history of commits.

6.2 Core Git Concepts

Working Directory: The local folder where files are actively edited.

Staging Area: A holding area where changes are placed before being committed.

Local Repository: The committed history stored on the developer's own machine.

Remote Repository: A shared copy of the repository hosted on a server such as GitHub.

.git: The hidden folder that stores all repository metadata and history.

.gitignore: Lists files and folders (e.g., .env, __pycache__/) that Git should not track.

README.md: A Markdown file describing the project, its setup, and usage instructions.



Figure 3: Git Workflow (Working Directory to Remote Repository)

7. Git Branching Strategy

This project follows a simplified feature-branch workflow built around two long-lived branches — main and develop — with short-lived feature branches created for individual pieces of work.

```

main
|
develop
|
|-- feature/login
|-- feature/sensor-monitoring
|-- feature/dashboard
|-- feature/alerts
`-- feature/database
  
```

main always contains stable, production-ready code. develop integrates completed features before they are considered stable. Each feature branch (for example feature/sensor-monitoring) is created from develop, developed in isolation, and merged back into develop once complete. In addition to feature branches, the project also defines:

- Release branch — used to prepare and stabilize a version before merging it into main.
- Hotfix branch — used to urgently patch a defect discovered in the production (main) branch.

7.1 Example Commands

```
git checkout develop
git checkout -b feature/sensor-monitoring
git add .
git commit -m "Add sensor monitoring API"
git checkout develop
git merge feature/sensor-monitoring
```

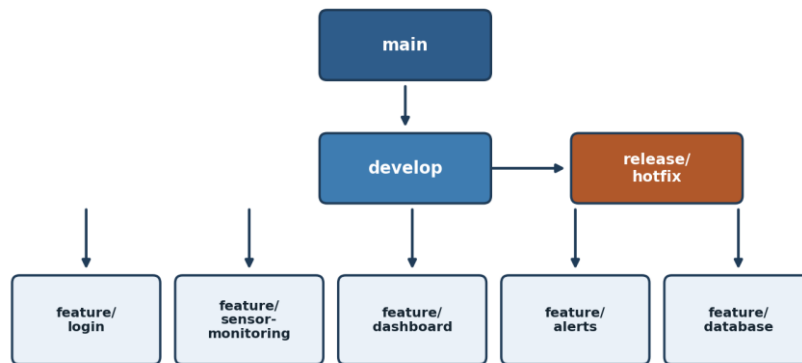


Figure 4: Git Branching Strategy

7.2 Why This Strategy Fits the Project

The project has several independent functional areas — authentication, sensor monitoring, the dashboard, alerting, and database work — each of which maps naturally to its own feature branch. This lets different pieces of functionality be developed and tested in isolation without destabilizing the shared develop branch, and it keeps main permanently deployable. The strategy also scales well: additional feature branches (for example feature/pond-management) can be added later without any change to the underlying workflow.

8. Version Control Workflow

8.1 Common Git Commands

- git status** — Shows changed, staged, and untracked files.
- git add** — Moves changes from the working directory into the staging area.
- git commit** — Records staged changes as a new point in the project history.
- git branch** — Lists, creates, or deletes branches.
- git checkout** — Switches between branches or restores files.
- git merge** — Combines the changes from one branch into another.
- git pull** — Fetches and merges changes from a remote repository.
- git push** — Uploads local commits to a remote repository.
- git log** — Displays the commit history.
- git diff** — Shows line-by-line differences between file versions.

8.2 Complete Workflow

```

Working Directory
| git add
v
Staging Area
| git commit
v
Local Repository
| git push
v
Remote Repository

```

8.3 Merge Conflicts

A merge conflict occurs when Git cannot automatically reconcile changes made to the same lines of a file on two different branches. When this happens, Git pauses the merge and marks the conflicting sections in the affected files using conflict markers (<<<<<<<<, =====, >>>>>>>>). The developer must manually review both versions, decide on the correct final content, remove the conflict markers, stage the resolved file with git add, and complete the merge with git commit. Clear commit messages and small, frequent commits reduce both the likelihood and the difficulty of resolving merge conflicts.

9. Commit History Analysis

A realistic commit history for this project, viewed with git log --oneline, would resemble the following:

```

a82f3d1 Add Docker Compose configuration
7c51e21 Add water quality alert system
4fa9d32 Add sensor monitoring API
2bd8a10 Create aquaculture database schema
19ce721 Add monitoring dashboard
0e431a2 Initial project setup

```

Meaningful commit messages make the project history readable and useful. A well-written message tells reviewers and future contributors what changed and why, without requiring them to inspect the full diff. This is especially valuable when tracing the origin of a bug, writing release notes, or onboarding a new team member.

9.1 Good vs. Bad Commit Messages

Good:

```
Add pH threshold alert service
```

Bad:

```
updated files
```

The good example clearly states what was added and its purpose, while the bad example is vague and gives no indication of what actually changed, forcing anyone reviewing the history to inspect the code directly.

10. Docker Environment Design

Docker is a containerization platform that packages an application together with its dependencies, libraries, and configuration into a single, portable unit called a container. Unlike a virtual machine, a container shares the host operating system's kernel, which makes it lightweight and fast to start while still providing process and filesystem isolation.

Docker is well suited to this project because it involves several interdependent services — a frontend, a backend API, and a MySQL database — each with its own runtime requirements. Without containerization, every developer or deployment target would need to manually install and configure matching versions of Python, MySQL, and web-server software, which is time-consuming and error-prone. Docker removes this burden: each service runs in a predictable, pre-configured environment that behaves identically on any machine that has Docker installed.

10.1 Proposed Containers

1. Frontend container — serves the static dashboard using nginx.
2. Backend container — runs the Flask REST API and alert engine.
3. MySQL database container — stores all persistent application data.
4. Sensor simulator container (optional) — generates simulated sensor readings for testing.

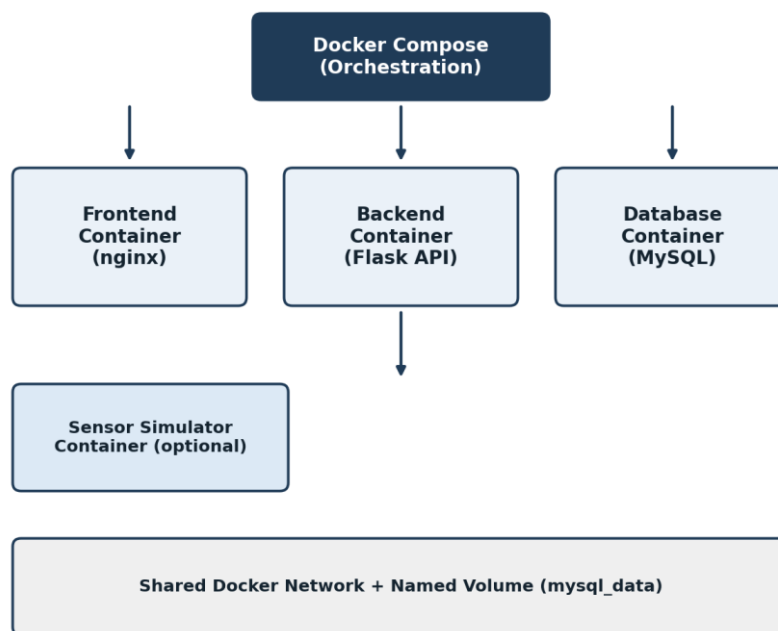


Figure 5: Docker Container Architecture

All containers are connected through a shared Docker network defined in `docker-compose.yml`, which allows them to reach each other by service name (for example, the backend connects to the database container using the hostname `database`). The MySQL container uses a named volume to persist data outside the container's writable layer, so that data is not lost when the container is recreated.

11. Dockerfile Development

The backend container is built from the following Dockerfile:

```
FROM python:3.11-slim
```

```

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

```

Table 6: Dockerfile Instruction Explanation

Instruction	Purpose
FROM	Specifies the base image (a lightweight Python 3.11 runtime) that the container is built on.
WORKDIR	Sets /app as the working directory for all subsequent instructions inside the container.
COPY	Copies files from the host machine into the container's filesystem.
RUN	Executes a command during the image build — here, installing Python dependencies.
EXPOSE	Documents that the container listens on port 5000.
CMD	Defines the default command executed when the container starts.

Copying requirements.txt and installing dependencies before copying the rest of the application code allows Docker to cache the dependency-installation layer. As long as requirements.txt does not change, subsequent builds reuse this cached layer, which significantly speeds up rebuilds during development.

12. Docker Compose Design

Docker Compose allows all of the project's containers to be defined and started together using a single YAML configuration file:

```

services:

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DB_HOST: database
      DB_NAME: fisheries
      DB_USER: fisheries_user
      DB_PASSWORD: fisheries_pass
    depends_on:
      - database

  database:
    image: mysql:8.0
    environment:

```

```

MYSQL_DATABASE: fisheries
MYSQL_USER: fisheries_user
MYSQL_PASSWORD: fisheries_pass
MYSQL_ROOT_PASSWORD: rootpass

```

```

ports:
  - "3306:3306"

```

```

volumes:
  - mysql_data:/var/lib/mysql

```

```

frontend:
  image: nginx:alpine
  ports:
    - "8080:80"
  volumes:
    - ./frontend:/usr/share/nginx/html
  depends_on:
    - backend

```

```

volumes:
  mysql_data:

```

Table 7: Docker Compose Configuration Elements

Element	Explanation
services	Each service (backend, database, frontend) defines one container to be built or pulled and run.
ports	Maps a container port to a host port so the service can be reached from outside Docker.
environment	Passes configuration values (database name, credentials, host) into the container at runtime.
depends_on	Controls startup order — e.g., the backend starts only after the database service is initiated.
volumes	mysql_data persists database files outside the container so data survives container restarts.
networks	Compose automatically creates a shared network so services can reach each other by service name.

With this configuration, running a single command builds and starts the frontend, backend, and database containers together, wires them onto a shared network, and mounts the persistent database volume — replacing what would otherwise be several manual setup steps on every machine.

13. Container Deployment and Testing

13.1 Deployment Commands

```

docker compose build
docker compose up -d
docker compose ps
docker compose logs
docker compose logs backend
docker compose down

```


docker compose build — Builds (or rebuilds) the images for all services defined in docker-compose.yml.

docker compose up -d — Starts all containers in detached (background) mode.

docker compose ps — Lists the status of all running containers in the project.

docker compose logs — Displays combined log output from all containers.

docker compose logs backend — Displays log output for the backend service only.

docker compose down — Stops and removes the containers, network, (but not named volumes by default).



Figure 7: Deployment Workflow

13.2 Expected Application Flow

```

graph TD
    Browser --> Frontend
    Frontend --> Backend_API[Backend API]
    Backend_API --> Database
  
```

13.3 Testing Plan

The following test cases describe the checks that would be carried out once containers are deployed. Screenshot evidence for each test is expected to be captured by the student and inserted at the marked placeholders; no tests have been executed as part of preparing this report.

Table 8: Container Testing Plan

Test Case	Expected Result	Status
Dashboard loading	Dashboard page loads successfully in the browser at the mapped port.	Pending
Backend API response	API endpoints return valid JSON responses with correct status codes.	Pending
Database connection	Backend successfully connects to the MySQL container on startup.	Pending
Sensor data insertion	Simulated sensor readings are stored correctly in the database.	Pending
Alert generation	Alert is raised when a simulated reading crosses a defined threshold.	Pending
Container status	All containers show a healthy/running state in docker compose ps.	Pending

[Insert Screenshot: Docker Compose containers running]

[Insert Screenshot: Git commit history]

[Insert Screenshot: Application dashboard]

14. Architecture and Component Interaction

Sensor Layer: Collects environmental data — temperature, pH, dissolved oxygen, and turbidity — from the (simulated) sensors.

Sensor Gateway: Transfers sensor readings from the sensor layer to the backend over the network.

Backend API: Receives, validates, processes, and stores incoming sensor information.

Alert Engine: Checks incoming sensor values against predefined safe threshold ranges and raises alerts when they are exceeded.

Database: Stores users, ponds, sensor readings, feeding records, and alerts.

Dashboard: Displays current and historical information to farm staff.

Administrator / Farmer: Uses the dashboard to monitor conditions and manage the aquaculture environment.

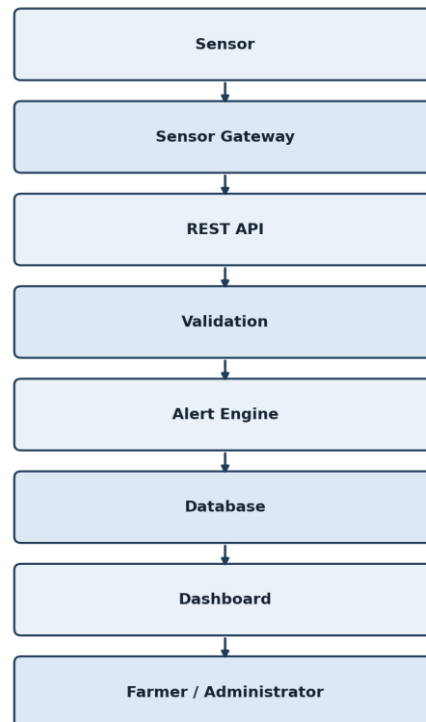


Figure 6: Data Flow and Component Interaction

Data flows in one direction from the sensor layer through validation and alerting to storage, and is then read back out through the dashboard. Structuring the interaction this way ensures that every reading is validated and evaluated for alerts before it is persisted, so that the stored data and any generated alerts remain consistent with one another.

15. Quality Attribute Analysis

Table 9: Quality Attribute Analysis

Quality Attribute	Implementation	Benefit
Performance	Lightweight Flask API and indexed MySQL queries.	Fast dashboard response times.
Scalability	Stateless backend containers behind Docker Compose.	Additional containers/ponds can be added easily.
Security	Environment variables for credentials; authenticated dashboard access.	Reduces exposure of sensitive configuration.
Reliability	Alert engine validates data before storage.	Consistent, trustworthy stored readings.
Availability	Independent containers restart without affecting others.	Reduced downtime from isolated failures.
Maintainability	Layered architecture with modular Git branches.	Features can be updated independently.
Portability	Dockerized services run identically across machines.	Easy setup on any Docker-capable host.
Usability	Simple, clearly labeled dashboard views.	Accessible to non-technical farm staff.
Testability	Modular routes/services and dedicated tests/ folder.	Components can be tested in isolation.

Git and Docker directly support several of these attributes: Git's branching model isolates in-progress feature work so that maintainability and testability are preserved on the shared develop branch, while Docker's container isolation and reproducible images improve portability, reliability, and availability by ensuring every environment — development, testing, or eventual production — runs the same configuration.

16. Benefits of Git and Docker

16.1 Benefits of Git

- Collaboration — multiple contributors can work on different features simultaneously without overwriting each other's work.
- Version Control — every change is recorded, so the project's evolution can be reviewed at any time.
- Branching — new features can be developed in isolation without destabilizing the main codebase.
- Backup — the full project history exists in every clone of the repository, not just on one machine.
- Change Tracking — it is always possible to see who changed what, and when.
- Rollback — problematic changes can be reverted to a previous known-good state.
- Code Review — pull requests allow changes to be reviewed before merging.
- Maintenance — a clear commit and branch history makes long-term maintenance easier.

16.2 Benefits of Docker

- Portability — containers run identically across development, testing, and production machines.
- Dependency Management — each service bundles its exact required dependencies.
- Isolation — services run independently, reducing the risk of one component affecting another.

- Reproducibility — the same Dockerfile and Compose file always produce the same environment.
- Easy Deployment — the entire application stack starts with a single Compose command.
- Scalability — additional container instances can be added to handle increased load.
- Maintainability — services can be updated or replaced independently.
- Environment Consistency — eliminates "it works on my machine" configuration issues.

16.3 Comparison

Table 10: Comparison – Without vs. With Git/Docker

Aspect	Without Git/Docker	With Git/Docker
Collaboration	Manual file sharing; risk of overwriting work.	Parallel work via branches with a merge-based workflow.
Environment Setup	Manual installation of Python, MySQL, and web server on each machine.	One command builds and runs the full environment.
Change History	No formal history; changes are hard to trace.	Complete, searchable commit history.
Deployment	Inconsistent between machines.	Consistent, reproducible container-based deployment.
Recovery from Errors	Difficult to revert to a previous working state.	Easy rollback via Git history or container image versions.

17. Challenges and Improvements

17.1 Realistic Challenges

- Merge conflicts when multiple feature branches modify the same files.
- Dependency version conflicts between local development and container environments.
- Misconfiguration of Docker networking or environment variables.
- Port conflicts when multiple services attempt to bind the same host port.
- Ensuring database persistence across container rebuilds.
- Managing environment variables and secrets securely.
- Simulated sensor connectivity not reflecting real-world network variability.
- Handling network failures gracefully between the gateway and backend.
- Processing sensor data in near real time without overloading the backend.

17.2 Proposed Improvements

- Introduce GitHub Actions for continuous integration and automated deployment (CI/CD).
- Add automated unit and integration tests that run on every pull request.
- Add Docker health checks so unhealthy containers are automatically restarted.
- Adopt a secure secrets-management approach instead of plain .env files in production.
- Use MQTT for more efficient, real-time sensor-to-gateway communication.
- Deploy the system to a cloud platform for wider accessibility.
- Apply machine learning to detect early warning patterns in water-quality trends.

- Build predictive fish-health analytics from historical sensor data.
- Add mobile push notifications for critical alerts.
- Explore automatic feeding control triggered by system logic.
- Introduce container orchestration (e.g., Kubernetes) for larger-scale deployment.

18. Results and Expected Outcomes

This section distinguishes between the expected results of the design presented in this report and the actual results/screenshots that the student will capture after implementing and running the project.

18.1 Expected Results

- Successful repository management with a clear, organized Git history.
- A working branching strategy separating main, develop, and feature branches.
- Containerized backend, database, and frontend services.
- Successful multi-container deployment using Docker Compose.
- Dashboard-based monitoring of simulated sensor data.
- Reliable storage of sensor data in the MySQL database.
- Automatic alert generation for out-of-range readings.

18.2 Actual Results

[This subsection is reserved for the student to insert actual screenshots and observations captured after running the system, corresponding to the placeholders provided in Section 13.]

19. Conclusion

The Smart Fisheries Monitoring and Aquaculture Management System demonstrates how continuous, sensor-driven monitoring can improve aquaculture management by replacing infrequent manual checks with a consistent stream of environmental data. Sensors provide raw environmental information, the backend processes and validates this information, the database preserves a historical record for trend analysis, and the dashboard turns that record into actionable insight for farm decision-making.

Beyond the monitoring functionality itself, this assignment highlights the practical value of two software engineering practices. Git improves version management by giving every change a clear history and allowing multiple contributors to develop features in parallel through a disciplined branching strategy, without destabilizing a shared, stable codebase. Docker improves deployment and portability by packaging each service — frontend, backend, and database — into a reproducible container, removing environment-specific setup problems and enabling the entire system to be started with a single command through Docker Compose.

Together, the layered software architecture, structured Git workflow, and containerized deployment strategy presented in this report form a maintainable and scalable foundation. While the current implementation relies on simulated sensor data, the architecture is designed so that real IoT hardware, cloud deployment, and further analytical capabilities could be integrated in future iterations with minimal structural change.

20. References

[1] Git. (2024). Git Documentation. Retrieved from <https://git-scm.com/doc>

- [2] GitHub, Inc. (2024). GitHub Docs. Retrieved from <https://docs.github.com>
- [3] Docker, Inc. (2024). Docker Documentation. Retrieved from <https://docs.docker.com>
- [4] Docker, Inc. (2024). Docker Compose Documentation. Retrieved from <https://docs.docker.com/compose/>
- [5] Python Software Foundation. (2024). Python 3 Documentation. Retrieved from <https://docs.python.org/3/>
- [6] Oracle Corporation. (2024). MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc/>
- [7] OASIS. (2024). MQTT Version 5.0 Specification. Retrieved from <https://mqtt.org/>
- [8] Food and Agriculture Organization of the United Nations. (2022). The State of World Fisheries and Aquaculture. Retrieved from <https://www.fao.org/fishery/en>
- [9] Chowdury, M. et al. (2021). IoT Based Smart Aquaculture Monitoring System: A Review. International Journal of Engineering Research and Technology.

Rubric Alignment

The following table maps each evaluation criterion of the 25-mark Software Architecture rubric to the corresponding section(s) of this report.

Table 11: Rubric Alignment

Evaluation Criterion	Marks	Report Section
System Requirement Analysis	5	Section 2 (Problem Analysis) + Section 3 (System Requirements)
Selection of Software Architecture	5	Section 5 (Software Architecture)
Architecture Diagram and Component Design	5	Section 5 (Figure 1) + Section 4 (Project Structure)
Component Interaction and Quality Attribute Analysis	5	Section 14 (Component Interaction) + Section 15 (Quality Attributes)
Architectural Justification and Technical Presentation	5	Section 5 (Justification) + Sections 6–13 (Git & Docker)
TOTAL	25	

- System Requirement Analysis is addressed through the problem statement, functional and non-functional requirement tables, and hardware/software requirement listings.
- Selection of Software Architecture is justified in Section 5 by explaining why a layered architectural style fits the natural data flow of a monitoring system.
- Architecture Diagram and Component Design is supported by the layered architecture diagram, the repository structure, and their accompanying explanations.
- Component Interaction and Quality Attribute Analysis is covered through the data-flow diagram and the quality-attribute table linking each attribute to its implementation and benefit.
- Architectural Justification and Technical Presentation is demonstrated through the consistent use of Git version control and Docker containerization to realize and deploy the proposed architecture.